



10.1. Operations on Queues

[Previous Section:](#)

 [10. Queues](#)

[Contents:](#)

[1. Basic Operations](#)

[2. Queue Implementation Using Python Lists](#)

[3. Queue Implementation Using Linked Lists](#)

[Summary](#)

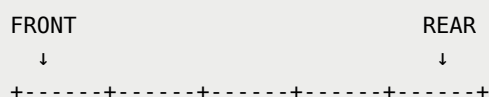
[References](#)

In the previous section, we learned that a queue follows the **First In First Out (FIFO)** principle. Elements enter from the **rear** and leave from the **front**, just like people waiting in a line.

However, simply storing data inside a queue is not enough. We must also be able to manipulate the queue efficiently. To do that, we perform several fundamental operations that allow us to insert elements, remove elements, inspect the queue, and determine its current state.

These operations form the foundation of every queue implementation, regardless of whether the queue is built using arrays, Python lists, linked lists, or more advanced structures.

Before diving into the implementations, let us visualize a simple queue:



```
| A | B | C | D | E |  
+---+---+---+---+---+
```

In this queue:

- A will be removed first.
- E was inserted most recently.
- New elements will be inserted after E.
- Elements will always be removed from A onward.

1. Basic Operations

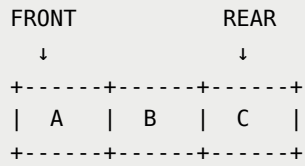
To effectively manage a queue, we need several essential operations:

Operation	Description	Time Complexity
ENQUEUE	Insert an element at the rear of the queue	$O(1)$
DEQUEUE	Remove an element from the front of the queue	$O(1)$
TRAVERSE	Visit every element in the queue	$O(n)$
GET FRONT (PEEK)	Return the front element without removing it	$O(1)$
GET REAR	Return the rear element without removing it	$O(1)$
GET SIZE	Return the number of elements in the queue	$O(1)$
IS EMPTY	Check whether the queue contains any elements	$O(1)$

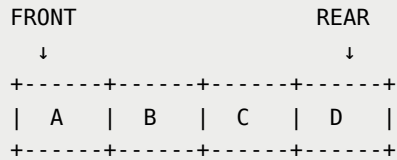
Let us examine each operation individually.

ENQUEUE

The **enqueue** operation inserts a new element at the rear of the queue. Suppose our queue contains:



After enqueueing D:



Notice that:

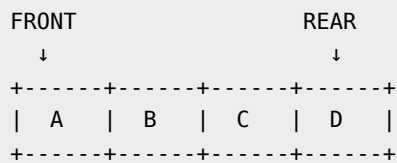
- FRONT remains unchanged.
- REAR moves to the newly inserted element.

Because insertion occurs directly at the rear, enqueue is considered an efficient operation.

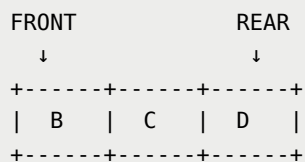
DEQUEUE

The **dequeue** operation removes the element at the front of the queue.

Suppose our queue contains:



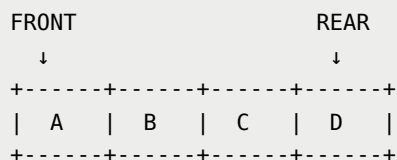
After dequeueing:



Element A is removed because it entered the queue first. This behavior is what makes queues FIFO structures.

TRAVERSE

Traversal means visiting every element in the queue from front to rear. For example:

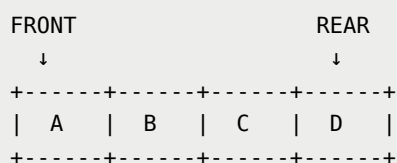


Traversing the queue produces: **A B C D**.

Traversal is useful when displaying or processing all elements stored in the queue.

GET FRONT (PEEK)

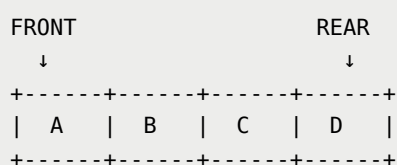
Sometimes we want to know which element will be removed next without actually removing it. The peek operation returns the front element.



Result: **A**. The queue remains unchanged.

GET REAR

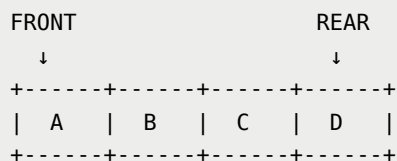
This operation returns the most recently inserted element.



Result: **D**. Again, the queue remains unchanged.

GET SIZE

This operation returns the total number of elements currently stored in the queue.

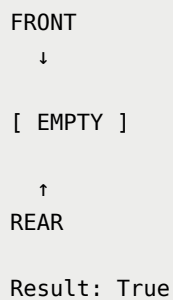


Result: **4**

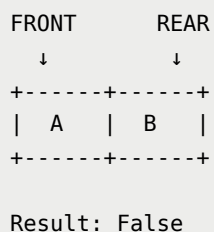
IS EMPTY

This operation determines whether the queue contains any elements.

- Empty queue:



- Non-empty queue:



2. Queue Implementation Using Python Lists

The simplest implementation of a queue uses a Python list.

In this approach:

- The beginning of the list acts as the FRONT.
- The end of the list acts as the REAR.

```

class Queue:
    def __init__(self): # Initial queue
        self.items = []

    def is_empty(self): # Check for empty queue
        return self.items == []

    def traverse(self): # Traverse a queue
        if self.items:
            for item in self.items:
                print(item, end=" ")
            print()
        else:
            print("Queue is empty")

    def enqueue(self, item): # Enqueue an item to the queue
        return self.items.insert(len(self.items), item)

    def dequeue(self): # Dequeue an item from the queue
        return self.items.pop(0)

    def get_front(self): # Get the front of the queue
        return self.items[0]

    def get_rear(self): # Get the rear of the queue
        return self.items[len(self.items) - 1]

    def get_size(self): # Size of the queue
        return len(self.items)

```

Queue visualization:

```

FRONT      REAR
  ↓         ↓
[A] → [B] → [C] → [D]

```

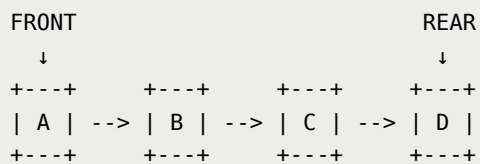
Although simple and easy to understand, removing the first element from a Python list requires shifting all remaining elements forward, making large

queues less efficient.

3. Queue Implementation Using Linked Lists

A more efficient implementation uses linked lists. Instead of shifting elements after every dequeue operation, we simply move the FRONT pointer to the next node.

Queue structure using a linked list:



- Node Class

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

- Queue Class

```
class Queue:
    def __init__(self): # Initial queue
        self.front = None
        self.rear = None
        self.size = 0

    def is_empty(self): # Check for empty queue
        return self.size == 0

    def traverse(self): # Traverse a queue
        if self.is_empty():
            print("Queue is empty")
            return
```

```

        current = self.front
    while current:
        print(current.data, end=" ")
        current = current.next
    print()

def enqueue(self, item): # Enqueue an item to the queue
    new_node = Node(item)

    if self.is_empty():
        self.front = self.rear = new_node
    else:
        self.rear.next = new_node
        self.rear = new_node

    self.size += 1

def dequeue(self): # Dequeue an item from the queue
    if self.is_empty():
        print("Queue is empty")
        return

    removed_data = self.front.data
    self.front = self.front.next

    self.size -= 1

    if self.is_empty():
        self.rear = None

    return removed_data

def get_front(self): # Get the front of the queue
    if self.is_empty():
        print("Queue is empty")
        return

```

```

        return self.front.data

    def get_rear(self): # Get the rear of the queue
        if self.is_empty():
            print("Queue is empty")
            return

        return self.rear.data

    def get_size(self): # Size of the queue
        return self.size

```

Compared to the list implementation, linked lists provide more efficient insertion and deletion because no shifting of elements is required.

Summary

In this section, we explored the fundamental operations of queues. We learned that every queue typically supports: Enqueue; Dequeue; Traverse; Get Front (Peek); Get Rear; Get Size; Is Empty. These operations allow us to insert, remove, inspect, and manage queue elements while maintaining the FIFO principle.

We also examined two common implementations: Python List-based Queue and Linked List-based Queue. While both implementations follow the same queue behavior, linked-list queues are generally more efficient for frequent insertions and deletions because elements do not need to be shifted after removal.

In the upcoming sections, we will explore two important variations of queues: the **Circular Queue** and the **Double-Ended Queue (Deque)**.

The Circular Queue improves upon the traditional linear queue by allowing the rear position to wrap around to the front, making better use of available space. The Double-Ended Queue extends the concept even further by allowing insertion and deletion operations at both ends of the queue, providing greater flexibility for a wide variety of applications.

You may have noticed that we have not included the **Priority Queue** here. While it is technically a type of queue, it introduces concepts that are significantly different from ordinary FIFO queues. More importantly, Priority Queues

naturally lead us to one of the most important data structures in computer science: the **Heap**.

Because of its importance and its close relationship with heaps, Priority Queue deserves its own dedicated section. There, we will not only learn how priority-based processing works, but also explore heap structures that allow Priority Queues to perform insertion and removal operations efficiently.

References

https://drive.google.com/file/d/1A7WberDJxGmGtXpGZ7v-FFATGRlgKoQZ/view?usp=drive_link

<https://www.geeksforgeeks.org/dsa/basic-operations-for-queue-in-data-structure/>

Next Section:

 [10.2. Circular Queues](#)